Programowanie w języku Kotlin

dr inż. Michał Sawicki
Politechnika Śląska

wersja: 15.10.2021

Literatura

kotlinlang.org
kotlinlang.org/docs/kotlin-docs.pdf

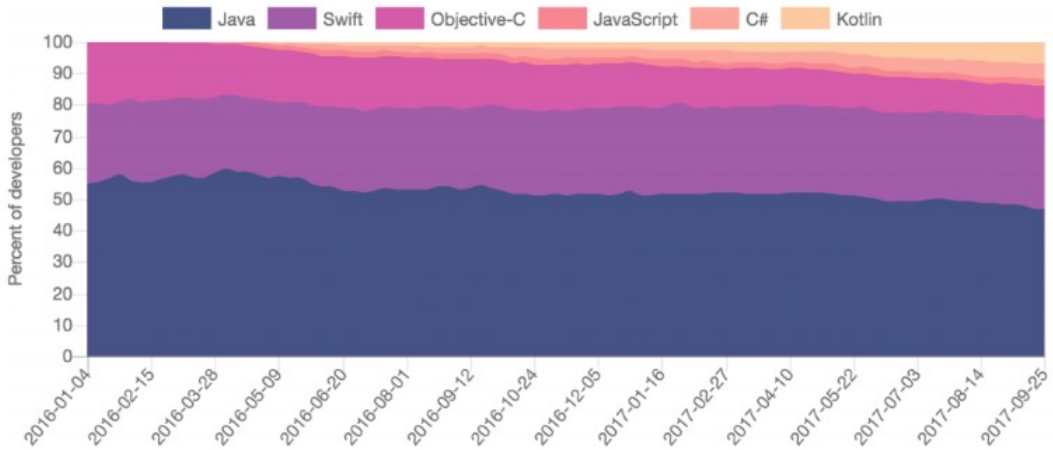
Nie tylko dinozaury...

What languages are mobile developers using?

The migration to modern languages is happening now

There are seven major languages for mobile platforms today, but the balance of power is quickly shifting. Not too long ago, Swift didn't exist. Now, it towers over Objective-C for iOS developers.

Android users are at the beginning of their own transition. With Java use peaking, Kotlin is on the path to winning the Android platform.



Java na Androidzie umiera. Deweloperzy, którzy nie będą się posługiwać Kotlinem, mogą wkrótce podzielić los dinozaurów...

Kotlin – ogólna charakterystyka

- Cel: **naprawa** problemów ze składnią Javy, **wzbogacenie** języka o rozwiązania znane z innych języków.
- Nie wymaga stosowania **średników** (średniki są opcjonalne).
- Kotlin jest kompatybilny z JDK6.
- Aplikacja napisana w Kotlinie działa **nie wolniej** niż aplikacja napisana w Javie.
- Kotlin zapewnia w pełni **interoperacyjność** z Javą, możliwe są rozwiązania **hybrydowe.**
- Istnieje **konwerter** kodu z Javy do Kotlina.
- Możliwa jest kompilacja kodu Kotlina do kodu **maszynowego,** nie pośredniego (z wykorzystaniem LLVM).

Kotlin – ogólna charakterystyka

- W Kotlinie nie ma metod, są za to **funkcje członkowskie** (lub prościej funkcje).
- Indeksy numerowane są **od zera** (jak w rodzinie języka C).
- Składowe klas domyślnie są **publiczne**.
- W Kotlinie możliwe jest **zagnieżdżanie** funkcji.
- Pliki z kodem Kotlina mają rozszerzenie **.kt**.
- Klasą bazową dla wszystkich klas jest **Any** (nie JObject).

Kotlin

- ✓ Obiektowy
- ✓ Strukturalny
- ✓ Funkcyjny
- ✓ Imperatywny
- ✓ Statycznie typowany

Kotlin – zastosowanie

- Aplikacje desktopowe
- Aplikacje webowe *(transkodowanie do JavaScript)*
- Aplikacje wbudowane
- Aplikacje mobilne

Warsztat programisty Kotlina

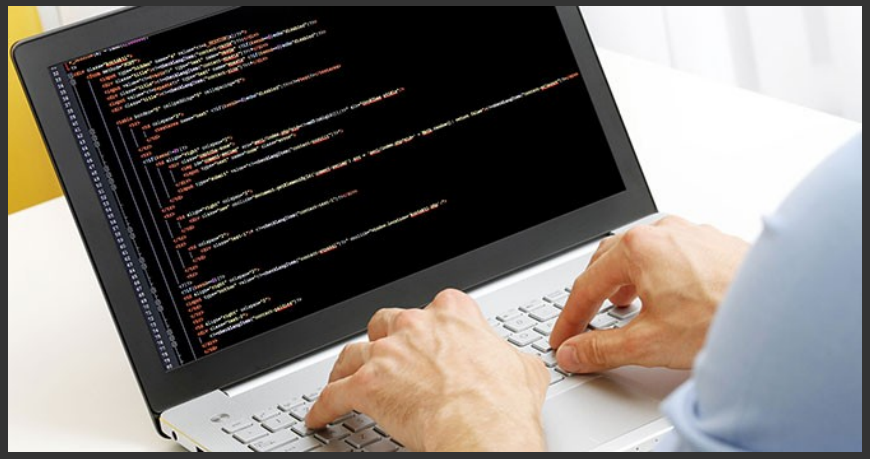
- Maszyna wirtualna Javy (**JVM**)
- Zintegrowane środowisko programistyczne (**IDE**)

→ IntelliJ IDEA

→ Eclipse *(wymaga dodatku)*

→ NetBeans *(wymaga dodatku)*

→ Online try.kotlinlang.org



Szkielet prostego programu

```
package pl.polsl

fun main(args :Array<String>) :Unit
{
    //cialo funkcji
}
```

`Unit` jest odpowiednikiem `void` i może być pomijany.

Witaj Kotlinie!

```
package pl.polsl

fun main(args:Array<String>)
{
    println("Witaj Kotlinie!")
}
```

Ciało blokowe funkcji

```
package pl.polsl

fun funkcja(): Int
{
    return 5
}
```

Ciało wyrażeniowe funkcji

```
package pl.polsl

fun funkcja1(): Int = 5

fun funkcja2() = 5
```

domniemanie typów

Argumenty funkcji

```
package pl.polsl

fun funkcja(a: Int, b: Int, c: Int)

funkcja(1,2,3)

funkcja(a=1,b=2,c=3)
```

Wiele argumentów: `vararg`

Podstawowe typy zmiennych

- Double (64)
- Float (32)
- Long (64)
- Int (32)
- Short (16)
- Byte (8)
- Char (*nie może być traktowany jako liczba*)
- Boolean (true, false)
- String

Przykłady wartości:

```
123
123L
0x0F
0b00110011  (nie ma zapisu oktalnego)
123.5
123.5e10
123.5f
```

W Kotlinie wszystko jest obiektem!

Deklaracja zmiennych

`val` – niemutowalna wartość zmiennej

`var` – mutowalna wartość zmiennej

```
val liczba: Int=5

val liczba=5

val liczba: Int
liczba=5

var liczba: Int=5
liczba=6

var liczba=5
liczba=6
```

Jeżeli zmienna (**typ zerowalny**) może przyjmować wartość **null** to musi to być wyraźnie zaznaczone poprzez `?` :

```
var zmienna :Typ?
```

Rzutowanie jawne

toByte()

toShort()

toInt()

toLong()

toFloat()

toDouble()

toChar()

```
var liczba:Int=66
println(liczba.toChar())
```

Tablice

- Tablice reprezentowane są przez generykę `Array<T>`, która zawiera funkcje `get`, `set`, `size` oraz przeciążony operator `[]`.
- Tworzenie tablic z wykorzystaniem funkcji fabrykujących `arrayOf()` lub `arrayOfNulls()` lub z wykorzystaniem konstruktora `Array(liczba, recepta_na_kolejne_wartości)`
- Oddzielne klasy dla typów podstawowych: `ByteArray`, `IntArray`… i odpowiednio funkcje fabrykujące `byteArrayOf`, `intArrayOf`…

```
var tab=intArrayOf(1,2)
var tab2=tab
tab[0]=5
print(tab2[0])
```

Operator rozciągania:
`*tab`

```
var tablica1:IntArray=intArrayOf(1,2,3)
println(tablica1[2])

var tablica2= intArrayOf(1,2,3)
println(tablica2.get(2))

var slownik=arrayOf("Ala","ma","kota")
println(slownik[1])

var liczby1=Array<Int>(10,{i->i})
println(liczby1.get(2))

var liczby2=Array(10,{i->i})
println(liczby2.get(2))
```

Łańcuchy znaków (String)

- Łańcuchy znaków są **niezmienne**.
- Elementami łańcucha są znaki typu Char i jest dostęp do nich poprzez operator [].
- Operatorem **konkatenacji** jest + (w tym rzutowanie niejawne, *uwaga na kolejność!*).
- Mogą zawierać znaki specjalne. Znak ucieczki to \ .
- **Surowy łańcuch** znaków z przejściami do nowych linii umieszczany jest w ” ” ” ” ” ” .
- **Szablony znakowe** zawierają znak $, po którym podawane jest wyrażenie.

Szablony znakowe

- Szablony znakowe (ze znakami specjalnymi lub surowe) zawierają znak $, po którym podawane jest wyrażenie.
- Analogiczny mechanizm do rozwiązania z Basha.

```
val i=10
println("i=$i")

val s="abc"
println("$s.length wynosi ${s.length}")

val cena1="\$9.99"
val cena2="""${'$'}9.99"""
```

Instrukcja warunkowa if

- Instrukcja warunkowa `if` jest wyrażeniem, tj. może zwracać wartość. (*Uwaga!* `try` *też może być wyrażeniem*)
- Nie ma operatora trynitarnego ? .
- Jeżeli instrukcja `if` jest traktowana jako wyrażenie musi zawierać blok `else`.
- Wyrażenie `if` może zawierać nietrywialne ciała (bloki), ale ostatnie wyrażenie w ciele jest wartością zwracaną przez blok.

```
if (1>0)
{
    print("zawsze prawda")
}

println(if(1>1){print("uwaga!")
                    "prawda"}
       else{"falsz"})
```

Instrukcja warunkowa when

- Odpowiednik instrukcji switch.
- Warunkiem nie musi być stała, może być wyrażeniem.
- Warunkiem może być weryfikacja typu.
- Może być traktowana jako wyrażenie.
- Alternatywa warunków za pomocą przecinka.

```
val a=0
println(when(a){
    0 -> "zero"
    0,1 -> "zero lub jeden"
    in 0..10 -> "liczba z 0 do 10"
    !in 20..30 -> "liczba poza 20 do 30"
    is Int -> "liczba typu Int"
    else -> "sam juz nie wiem"
})
```

Pętla for

- Odpowiednik instrukcji foreach.

```
for (i in 1..3)
{
    println(i)
}
for (i in 6 downTo 0 step 2)
{
    println(i)
}
```

Pętle while i do..while

- Działają jak w Javie.

```
var x=2
while(x>0)
{
    println(x)
    x--
}
do {
    println(x)
    x++
}while(x<2)
```

W Kotlinie mamy też `break` i `continue`.

Etykiety

- Wykorzystywane do określenia poziomu zagnieżdżenia.

```
petlaZewnetrzna@ for (i in 1..10)
{
    petlaWewnetrzna@ for (j in 2..10)
    {
        println(j)
        if (i==j)
        {
            break@petlaZewnetrzna
        }
    }
}
```

Lambda

```
{ /* argumenty */ -> /* cialo */ }

val suma = { x: Int, y: Int -> x+y }

{ it }

//Android - Java
button.setOnClickListener(new OnClickListener()
{
   @Override
   public void onClick(View arg)
   {
      //cialo
   }
});

//Android - Kotlin+lambda
button.setOnClickListener { /* cialo */ }
```

Klasa

- Słowo kluczowe `class`.
- Klasa może nie mieć ciała.
- Klasa domyślnie jest finalna, publiczna.
- Klasa może mieć **podstawowy konstruktor** i wiele alternatywnych konstruktorów.
- Ciało konstruktora podstawowego w bloku **init**.
- Konstruktory alternatywne znajdują się w ciele klasy.
- Operatory `==` i `===`
- Operator `is`

Konstruktor podstawowy

```
class Student constructor(nazwisko: String) { }

class Student(nazwisko: String) { }

class Student(nazwisko: String)
{
    val album=nazwisko
    init
    {
        println("Po raz pierwszy mam studenta $nazwisko")
    }
    val id=nazwisko
    init
    {
        println("Drugi raz ten sam student $nazwisko")
    }
}
```

widoczność

Konstruktor podstawowy – lista inicjalizacyjna

```
class Student(val nazwisko: String, var wiek: Int)
{
    //ciało klasy
}
```

Konstruktor alternatywny i delegacja do innych konstruktorów

```
class Student
{
    constructor(imie :String) {}
}
```

Delegacja do konstruktora podstawowego:

```
class Student(val nazwisko :String)
{
    constructor(nazwisko :String, imie :String) : this(nazwisko)
    {
        //ciało konstruktora alternatywnego
    }
}
```

Konstruktor alternatywny i delegacja do innych konstruktorów

```
class Student
{
    init
    {
        println("Podstawowy")
    }
    constructor(i: Int)
    {
        println("Alternatywny")
    }
}
```

Settery i gettery

```
class Student
{
    var liczba: Int
    get()
    {
        return liczba
    }
    set(n: Int)
    {
        liczba=n
    }
}
```

```
class Student
{
    var liczba: Int
    get()=liczba
}
```

Tworzenie obiektów klas

```
val student1=Student()

val student2=Student("Kowalski")
```

NULL!

```
obiekt?.funkcja()

obiekt1 ?: obiekt2

obiekt!!
```

operator Elvisa

asercja niezerowa

Dziedziczenie klas

```
open class Figura(nazwa:String) {}

class Kwadrat(nazwa:String, bok:Int) : Figura(nazwa) {}

class Trojkat : Figura
{
   constructor(nazwa:String) : super(nazwa) {}
}
```

`super` może być wykorzystane do wywołania funkcji klasy bazowej (lub zewnętrznej, możliwe etykiety)

Dziedziczenie klas – nadpisywanie funkcji

```
open class Figura
{
   open fun funkcja1() {}
   open fun funkcja2() {}
   fun funkcja3() {}
}

open class Kwadrat:Figura
{
   override fun funkcja1() {}
   final override funkcja2() {}
}
```

Dziedziczenie klas – nadpisywanie składników (pól)

```
open class Figura
{
   open val i:Int get() {//ciało funkcji get}
}

class Kwadrat:Figura
{
   override val i:Int=0
}

class Trojkat(override val i:Int):Figura {}

class Kolo:Figura
{
   override var i:Int=1
}
```

val → var

Implementacja interfejsu

```
interface IFigura
{
    fun funkcja()
}

class Kwadrat: IFigura
{
    override fun funkcja() {}
}
```

Widoczność składników

- **Publiczna** (`public`) – pola i metody dostępne na zewnątrz klasy
- **Wewnętrzna** (`internal`) – podobnie jak `public`, ale tylko wewnątrz modułu
- **Prywatna** (`private`) – pola i metody dostępne jedynie z wnętrza (ciała) klasy
- **Chronione** (`protected`) – pola i metody dostępne tylko z wnętrza klasy lub klasy pochodnej

Funkcja rozszerzająca

```
package strings

fun String.lastChar(): Char = this.get(this.length-1)
```

```
println("Politechnika".lastChar())
```

Klasa wyliczeniowa

```
enum class Plec {
 MEN, WOMEN
}
```

miękkie słowo kluczowe

Klasa wyliczeniowa:
- może zawierać funkcje

Klasa i obiekt w jednym

```
fun main(arg:Array<String>)
{
    Dyrektor.przedstawSie()
}

object Dyrektor
{
    private val nazwisko="Kowalski"
    fun przedstawSie()
    {
        println(nazwisko)
    }
}
```

Inne elementy Kotlina

- Klasy danych (`data class`)
- Klasy zamknięte (`sealed class`)
- Generyki *(analogicznie jak w Javie)*
- Operator `!==` oraz `::` - pobranie referencji funkcji
- Funkcje standardowe

Android lubi Kotlina

```
class MainActivity : Activity(), View.OnClickListener
{
    override fun onCreate(savedInstanceState: Bundle?)
    {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        val przycisk: Button =findViewById(R.id.button)
        przycisk.setOnClickListener(this)
    }
    override fun onClick(v: View?)
    {
        Toast.makeText(applicationContext,"Hello",Toast.LENGTH_SHORT).show()
    }
}
```